

Modelo de resolución de ejercicio: Memoria compartida

Consigna

Un BANCO posee 3 cajeros y un tesorero. Cada cajero y el tesorero están representados por un proceso independiente:

CAJERO1

CAJERO2

CAJERO3

TESORERO.

a) CAJEROS

Cada cajero genera un depósito en forma aleatoria con la siguiente información:

Importe – \$100 a \$1000 (aleatorio)

Cheque – Si o No (0 ó 1).

Los depósitos se van grabando en memoria compartida con la siguiente estructura (100 registros):

```
structtipo_dato
{
    int cajero;    //Numero del cajero
    int importe;  //importe
    int cheque;   //0=Efectivo 1=Cheque.
    int listo;    //0=Registro libre 1=Registro a procesar.
};
```

El cajero deberá buscar un registro libre (listo = 0) para cargar un depósito en memoria compartida. En caso de que no encuentre un registro libre, deberá reintentar hasta que encuentre uno libre.

b) TESORERO

El tesorero deberá revisar la lista de depósitos de los registros en memoria compartida. Al finalizar de procesar un registro, deberá colocar listo=0, indicando que ese registro está libre (ya procesado).

Por cada registro deberá ir actualizado los siguientes totalizadores y mostrarlos por pantalla:

TOTAL

TC-EFEC	T\$-EFEC	TC-CHEQ	T\$-CHEQ
10	\$100	5	\$1025

CAJERO1

TC-EFEC	T\$-EFEC	TC-CHEQ	T\$-CHEQ
2	\$50	1	\$1000

CAJERO2

TC-EFEC	T\$-EFEC	TC-CHEQ	T\$-CHEQ
3	\$20	2	\$20

CAJERO3

TC-EFEC	T\$-EFEC	TC-CHEQ	T\$-CHEQ
5	\$30	2	\$5

Importante:

- La solución debe estar desarrollada en 4 procesos, utilizando memoria compartida y utilizando semáforos.
- Los procesos de CAJEROS deben funcionar en forma automática simulando los depósitos con una espera ALEATORIA entre 1000ms y 2500ms.
- TESORERO posee un bucle que contrala si hay nuevos depósitos cada 500ms.

Notas:

- El programa debe estar INDENTADO CORRECTAMENTE.
- Utilizar DEFINE para constantes.

Extra: modificar para N cajeros.

Resolución

Tesorero:

```
#include <sys/shm.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
```

```

#include <sys/sem.h>
#define ROJO 0
#define VERDE 1

#define CANTIDAD 100

#define DESDE_IMPORTE 10
#define HASTA_IMPORTE 50

#define DESDE_CHEQUE 0
#define HASTA_CHEQUE 1

#define INTERVALO 5000

#define SEMAFORO0 0
#define CANT_SEMAFORO 1

#define CANT_MAX_CAJEROS 3

typedef struct tipo_dato dato;

struct tipo_dato
{
    int cajero;
    int importe;
    int cheque;
    int listo;
};

key_t creo_clave()
{
    key_t clave;
    clave = ftok("/bin/ls", 33);
    if (clave == (key_t)-1)
    {
        printf("No puedo conseguir clave para memoria compartida\n");
        exit(0);
    }
    return clave;
}

void *creo_memoria(int size, int *r_id_memoria)
{
    void *ptr_memoria;
    int id_memoria;

```

```

id_memoria = shmget(creo_clave(), size, 0777 | IPC_CREAT);

if (id_memoria == -1)
{
    printf("No consigo id para memoria compartida\n");
    exit(0);
}

ptr_memoria = (void *)shmat(id_memoria, (char *)0, 0);

if (ptr_memoria == NULL)
{
    printf("No consigo memoria compartida\n");
    exit(0);
}
*r_id_memoria = id_memoria;
return ptr_memoria;
}

//funcion que crea el semaforo
int creo_semaforo()
{
    key_t clave = creo_clave();
    int id_semaforo = semget(clave, 1, 0600 | IPC_CREAT);
    //PRIMER PARAMETRO ES LA CLAVE, EL SEGUNDO CANT SEMAFORO, EL TERCERO 060
    0 LO UTILIZA CUALQUIERA, IPC ES CONSTANTE (VEA SEMAFORO)
    if (id_semaforo == -1)
    {
        printf("Error: no puedo crear semaforo\n");
        exit(0);
    }
    return id_semaforo;
}

//inicia el semaforo
void inicia_semaforo(int id_semaforo, int valor)
{
    semctl(id_semaforo, 0, SETVAL, valor);
}

//levanta el semaforo
void levanta_semaforo(int id_semaforo)
{

```

```

    struct sembuf operacion;
    printf("Levanta SEMAFORO \n");
    operacion.sem_num = 0;
    operacion.sem_op = 1; //incrementa el semaforo en 1
    operacion.sem_flg = 0;
    semop(id_semaforo, &operacion, 1);
}

//espera semaforo
void espera_semaforo(int id_semaforo)
{
    struct sembuf operacion;
    printf("Espera SEMAFORO \n");
    operacion.sem_num = 0;
    operacion.sem_op = -1; //decrementa el semaforo en 1
    operacion.sem_flg = 0;
    semop(id_semaforo, &operacion, 1);
}

int main(int argc, char *argv[])
{
    dato *memoria = NULL;

    int id_memoria;
    int i, j, aleatorio;
    int id_semaforo;
    int posicion;
    int intervalo = INTERVALO;

    int total_cantidad_depositos = 0;
    int total_cantidad_efectivos = 0;
    int total_cantidad_cheques = 0;

    int total_pesos_depositos = 0;
    int total_pesos_efectivos = 0;
    int total_pesos_cheques = 0;

    int vttotal_cantidad_depositos[CANT_MAX_CAJEROS] = {0};
    int vttotal_cantidad_efectivos[CANT_MAX_CAJEROS] = {0};
    int vttotal_cantidad_cheques[CANT_MAX_CAJEROS] = {0};

    int vttotal_pesos_depositos[CANT_MAX_CAJEROS] = {0};
    int vttotal_pesos_efectivos[CANT_MAX_CAJEROS] = {0};
    int vttotal_pesos_cheques[CANT_MAX_CAJEROS] = {0};

```

```

if (argc > 1)
{
    intervalo = atoi(argv[1]);
}
//cambia la semilla para random, usa el time como semilla inicial
srand(time(NULL));

id_semaforo = creo_semaforo();

inicia_semaforo(id_semaforo, VERDE);

memoria = (dato *)creo_memoria(sizeof(dato) * CANTIDAD, &id_memoria);

//Inicia memoria.
for (i = 0; i < CANTIDAD; i++)
    memoria[i].listo = 0;

while (1)
{
    //printf("Tesorero\n");
    espera_semaforo(id_semaforo);
    i = 0;
    while (i < CANTIDAD)
    {
        if (memoria[i].listo == 1)
        {
            if (memoria[i].cajero <= CANT_MAX_CAJEROS)
            {
                memoria[i].listo = 0;

                total_cantidad_depositos++;
                total_pesos_depositos += memoria[i].importe;

                vttotal_cantidad_depositos[memoria[i].cajero - 1]++;
                vttotal_pesos_depositos[memoria[i].cajero - 1] += memoria
[i].importe;

                if (memoria[i].cheque == 0)
                {
                    total_cantidad_efectivos++;
                    total_pesos_efectivos += memoria[i].importe;
                    vttotal_cantidad_efectivos[memoria[i].cajero - 1]++;
                    vttotal_pesos_efectivos[memoria[i].cajero - 1] += mem
oria[i].importe;
                }
            }
        }
        i++;
    }
}

```

```

        else
        {
            total_cantidad_cheques++;
            total_pesos_cheques += memoria[i].importe;
            vttotal_cantidad_cheques[memoria[i].cajero - 1]++;
            vttotal_pesos_cheques[memoria[i].cajero - 1] += memoria[i].importe;
        }
        system("clear");
        printf("TCD %d \nT$D %d \nTCE %d \nT$E %d \nTCC %d \nT$C %d \n\n", total_cantidad_depositos,
            total_pesos_depositos,
            total_cantidad_efectivos,
            total_pesos_efectivos,
            total_cantidad_cheques,
            total_pesos_cheques);

        for (j = 0; j < CANT_MAX_CAJEROS; j++)
        {
            printf("Cajero %d\n", j + 1);
            printf("TCD %d \nT$D %d \nTCE %d \nT$E %d \nTCC %d \nT$C %d \n\n", vttotal_cantidad_depositos[j],
                vttotal_pesos_depositos[j],
                vttotal_cantidad_efectivos[j],
                vttotal_pesos_efectivos[j],
                vttotal_cantidad_cheques[j],
                vttotal_pesos_cheques[j]);
        }
    }
    i++;
};
levanta_semaforo(id_semaforo);
usleep(intervalo * 1000);
}
shmdt((char *)memoria);
shmctl(id_memoria, IPC_RMID, (struct shmid_ds *)NULL);
return 0;
}

```

Cajeros:

```
#include <sys/shm.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#define ROJO 0
#define VERDE 1

#define CANTIDAD 100

#define DESDE_IMPORTE 10
#define HASTA_IMPORTE 50

#define DESDE_CHEQUE 0
#define HASTA_CHEQUE 1

#define DESDE_INTERVALO 1000
#define HASTA_INTERVALO 2500

#define SEMAFORO0 0
#define CANT_SEMAFORO 1

#define CAJERO 1

typedef struct tipo_dato dato;

struct tipo_dato
{
    int cajero;
    int importe;
    int cheque;
    int listo;
};

key_t creo_clave()
{
    key_t clave;
    clave = ftok("/bin/ls", 33);
    if (clave == (key_t)-1)
    {
        printf("No puedo conseguir clave para memoria compartida\n");
        exit(0);
    }
}
```



```

    }
    return clave;
}

void *creo_memoria(int size, int *r_id_memoria)
{
    void *ptr_memoria;
    int id_memoria;
    id_memoria = shmget(creo_clave(), size, 0777 | IPC_CREAT);

    if (id_memoria == -1)
    {
        printf("No consigo id para memoria compartida\n");
        exit(0);
    }

    ptr_memoria = (void *)shmat(id_memoria, (char *)0, 0);

    if (ptr_memoria == NULL)
    {
        printf("No consigo memoria compartida\n");
        exit(0);
    }
    *r_id_memoria = id_memoria;
    return ptr_memoria;
}

//funcion que crea el semaforo
int creo_semaforo()
{
    key_t clave = creo_clave();
    int id_semaforo = semget(clave, 1, 0600 | IPC_CREAT);
    //PRIMER PARAMETRO ES LA CLAVE, EL SEGUNDO CANT SEMAFORO, EL TERCERO 060
    //LO UTILIZA CUALQUIERA, IPC ES CONSTANTE (VEA SEMAFORO)
    if (id_semaforo == -1)
    {
        printf("Error: no puedo crear semaforo\n");
        exit(0);
    }
    return id_semaforo;
}

//inicia el semaforo

```

```

void inicia_semaforo(int id_semaforo, int valor)
{
    semctl(id_semaforo, 0, SETVAL, valor);
}

//levanta el semaforo
void levanta_semaforo(int id_semaforo)
{
    struct sembuf operacion;
    printf("Levanta SEMAFORO \n");
    operacion.sem_num = 0;
    operacion.sem_op = 1; //incrementa el semaforo en 1
    operacion.sem_flg = 0;
    semop(id_semaforo, &operacion, 1);
}

//espera semaforo
void espera_semaforo(int id_semaforo)
{
    struct sembuf operacion;
    printf("Espera SEMAFORO \n");
    operacion.sem_num = 0;
    operacion.sem_op = -1; //decrementa el semaforo en 1
    operacion.sem_flg = 0;
    semop(id_semaforo, &operacion, 1);
}

int main(int argc, char *argv[])
{
    dato *memoria = NULL;
    int id_memoria;
    int i, aleatorio;
    int id_semaforo;
    int importe, posicion, cheque;
    //Inicializa la cantidad por default.
    int cajero = CAJERO;

    if (argc > 1)
    {
        //toma la cantidad de cajeros por parametro.
        cajero = atoi(argv[1]);
    }
    printf("Cajero %d\n", cajero);

    //cambia la semilla para random, usa el time como semilla inicial

```

```

srand(time(NULL) - cajero);

id_semaforo = creo_semaforo();

memoria = (dato *)creo_memoria(sizeof(dato) * CANTIDAD, &id_memoria);

while (1)
{
    printf("Cajero\n");
    importe = rand() % (HASTA_IMPORTE - DESDE_IMPORTE + 1) + DESDE_IMPORTE;
TE;
    cheque = rand() % (HASTA_CHEQUE - DESDE_CHEQUE + 1) + DESDE_CHEQUE;
    espera_semaforo(id_semaforo);
    posicion = -1;
    i = 0;
    while (posicion == -1 && i < CANTIDAD)
    {
        if (memoria[i].listo == 0) //Cuando encuentra un registro
        {
            //Cambia el valor de posicion para salir del bucle.
            posicion = i;
            //carga la memoria.
            memoria[posicion].cajero = cajero;
            memoria[posicion].importe = importe;
            memoria[posicion].cheque = cheque;
            memoria[posicion].listo = 1;
            //Muestra por pantalla.
            printf("Posicion %d\n", posicion);
            printf("Cajero %d\n", memoria[posicion].cajero);
            printf("Importe %d\n", memoria[posicion].importe);
            printf("cheque %d\n", memoria[posicion].cheque);
            printf("listo %d\n", memoria[posicion].listo);
        }
        i++; //incrementa el contador.
    };
    levanta_semaforo(id_semaforo);
    usleep((rand() % (HASTA_INTERVALO - DESDE_INTERVALO + 1) + DESDE_INTERVALO) * 1000);
}
return 0;
}

```

Compilación y ejecución de los procesos:

```
//Compilación:
gcc -o tesorero tesorero.c
gcc -o cajeros cajeros.c

//Ejecución en distintas terminales:
//Terminal 1:
./tesorero

//Terminal 2:
./cajeros 1 15000

//Terminal 3:
./cajeros 2 15000

//Terminal 4:
./cajeros 3 15000
```